Issues with Hash Tables

Remember, we said that our earlier Hash Table implementation was naive. By that, we meant that the code doesn't cater to some frequently occurring issues. Let's see what these issues really are.

Collisions

A collision in a Hash Table occurs when an insert operation tries to insert an item at a table slot already occupied by another item. How can this ever happen? Let's find out.

Collision example

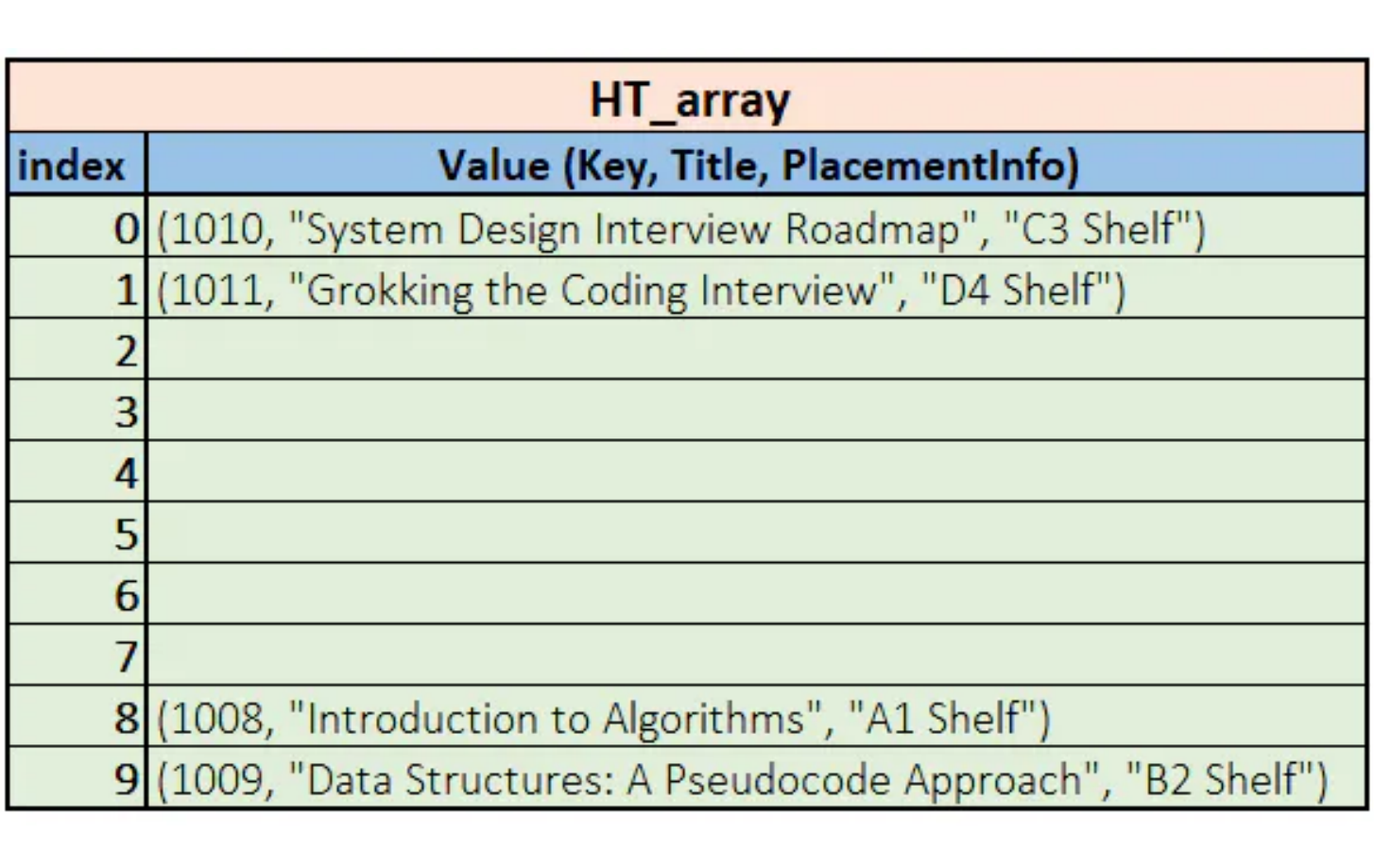
Reconsider our earlier example of the Hash Table for the public library book information storage. Assume, for the sake of simplicity, the Hash Table has the `max_length` equal to 10. Further, you need to insert the following book records in it:

Key	Title	Placement Info
1008	Introduction to Algorithms	A1 Shelf
1009	Data Structures: A Pseudocode Approach	B2 Shelf
1010	System Design Interview Roadmap	C3 Shelf
1011	Grokking the Coding Interview	D4 Shelf
1021	Grokking the Art of Recursion for Coding Interviews	E5 Shelf

Here is the hash value calculation for the first four entries:

Key	Hash Value
1008	$1008\%10 = 8$
1009	$1009\%10 = 9$
1010	$1010\%10 = 0$
1011	$1011\%10 = 1$

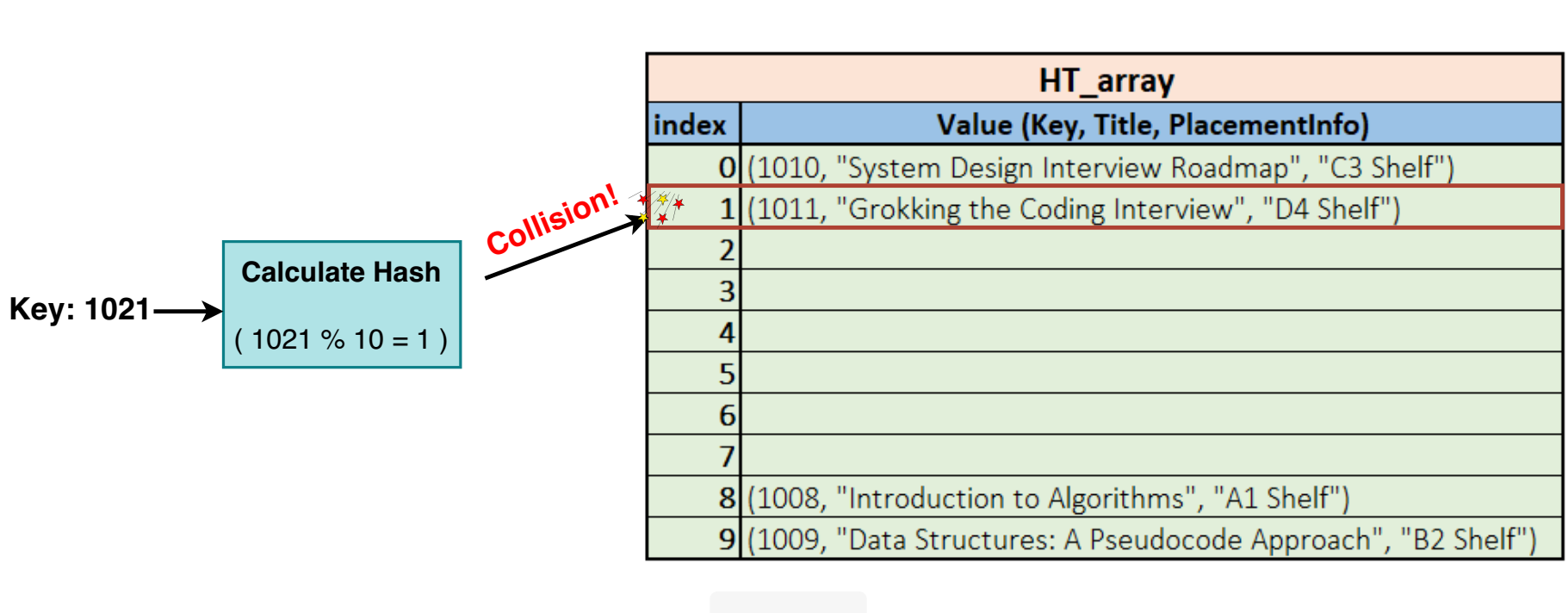
So the hash table array would look something like this:



HT_array	
index	Value (Key, Title, PlacementInfo)
0	(1010, "System Design Interview Roadmap", "C3 Shelf")
1	(1011, "Grokking the Coding Interview", "D4 Shelf")
2	
3	
4	
5	
6	
7	
8	(1008, "Introduction to Algorithms", "A1 Shelf")
9	(1009, "Data Structures: A Pseudocode Approach", "B2 Shelf")

Now, what happens if we try inserting the record with the `Key` 1021? The hash value for 1021 is the same as occupied by the book with the `Key` 1011. In this scenario, we say that a collision has occurred.

The phenomenon is depicted in the following illustration.



HT_array	
index	Value (Key, Title, PlacementInfo)
0	(1010, "System Design Interview Roadmap", "C3 Shelf")
1	(1011, "Grokking the Coding Interview", "D4 Shelf")
2	
3	
4	
5	
6	
7	
8	(1008, "Introduction to Algorithms", "A1 Shelf")
9	(1009, "Data Structures: A Pseudocode Approach", "B2 Shelf")

Collision

Collisions occur frequently in hash tables when two different keys hash to the same bucket. Without proper collision handling, lookup performance degrades from $O(1)$ to $O(n)$ linear search time. Managing collisions is crucial to efficient hash table implementation.

Overflows

Overflow in a hash table occurs when the number of elements inserted exceeds the fixed capacity allocated for the underlying bucket array. For example, if you already have inserted information of ten books in the earlier discussed hash table, inserting the 11^{th} one will cause an overflow.

One important point to note here is that as the underlying bucket array starts filling towards its maximum capacity, the expectation of collisions starts increasing. Thereby the overall efficiency of hash table operations starts decreasing.

An ideal hash table implementation must resolve collisions effectively and must act to avoid any overflow early. In the next section, we will explore different strategies for handling collisions.

Resolving Collisions

Remember that our naive hash table implementation directly overwrote existing records when collisions occurred. This is inaccurate as it loses data on insertion. This section describes how we can avoid such data losses with the help of collision resolution techniques.

Based on how we resolve the collisions, the collision resolution techniques are classified into two types:

- Open addressing / Closed hashing
- Chaining / Open hashing

Let's find out how these schemes help us resolve the collisions without losing any data.

Open Addressing (Closed Hashing):

Open addressing techniques resolve hash collisions by probing for the next available slot within the hash table itself. This approach is called open addressing since it opens up the entire hash table for finding an empty slot during insertion.

It is also known as closed hashing because the insertion is restricted to existing slots within the table without using any external data structures.

Depending on how you jump or probe to find the next empty slot, the closed hashing is further divided into multiple types. Here are the main collision resolution schemes used in the open-addressing scheme:

1. **Linear probing**: Linear probing is the simplest way to handle collisions by linearly searching consecutive slots for the next empty location.
2. **Quadratic probing**: When a collision occurs, the quadratic probing attempts to find an alternative empty slot for the key by using a `quadratic function` to probe the successive possible positions.
3. **Double hashing**: It uses two hash functions to determine the next probing location in case of a successive collision.
4. **Random probing**: Random probing uses a pseudo-random number generator (PRNG) to compute the step size or increment value for probes in case of collisions.

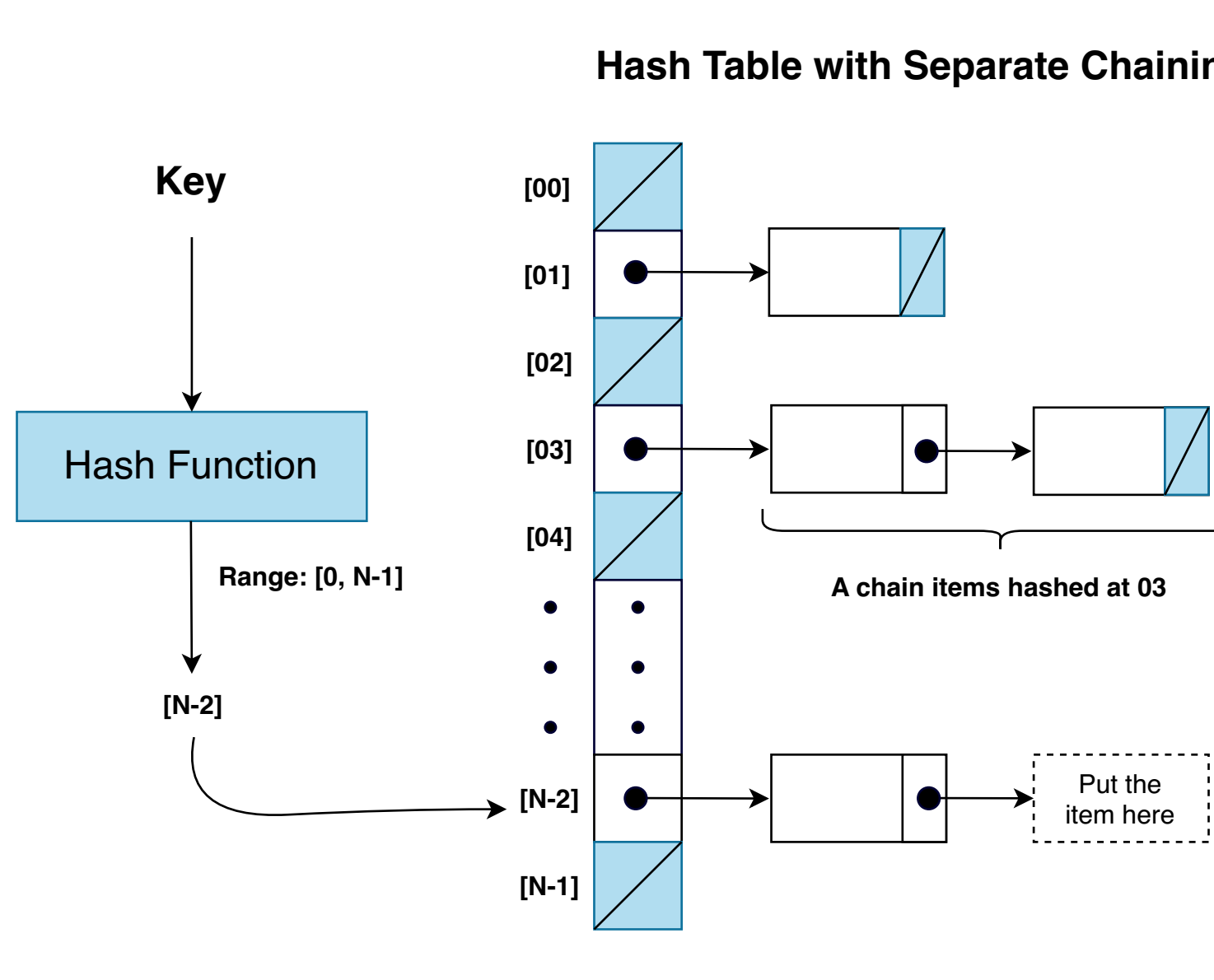
Implementation of insertion, search, and deletion operations is slightly different for each type of the operations.

Separate Chaining (Open Hashing):

Selecting the right closed hashing technique for resolving collisions can be tough. You need to keep the pros and cons of different strategies in mind and then have to make a decision.

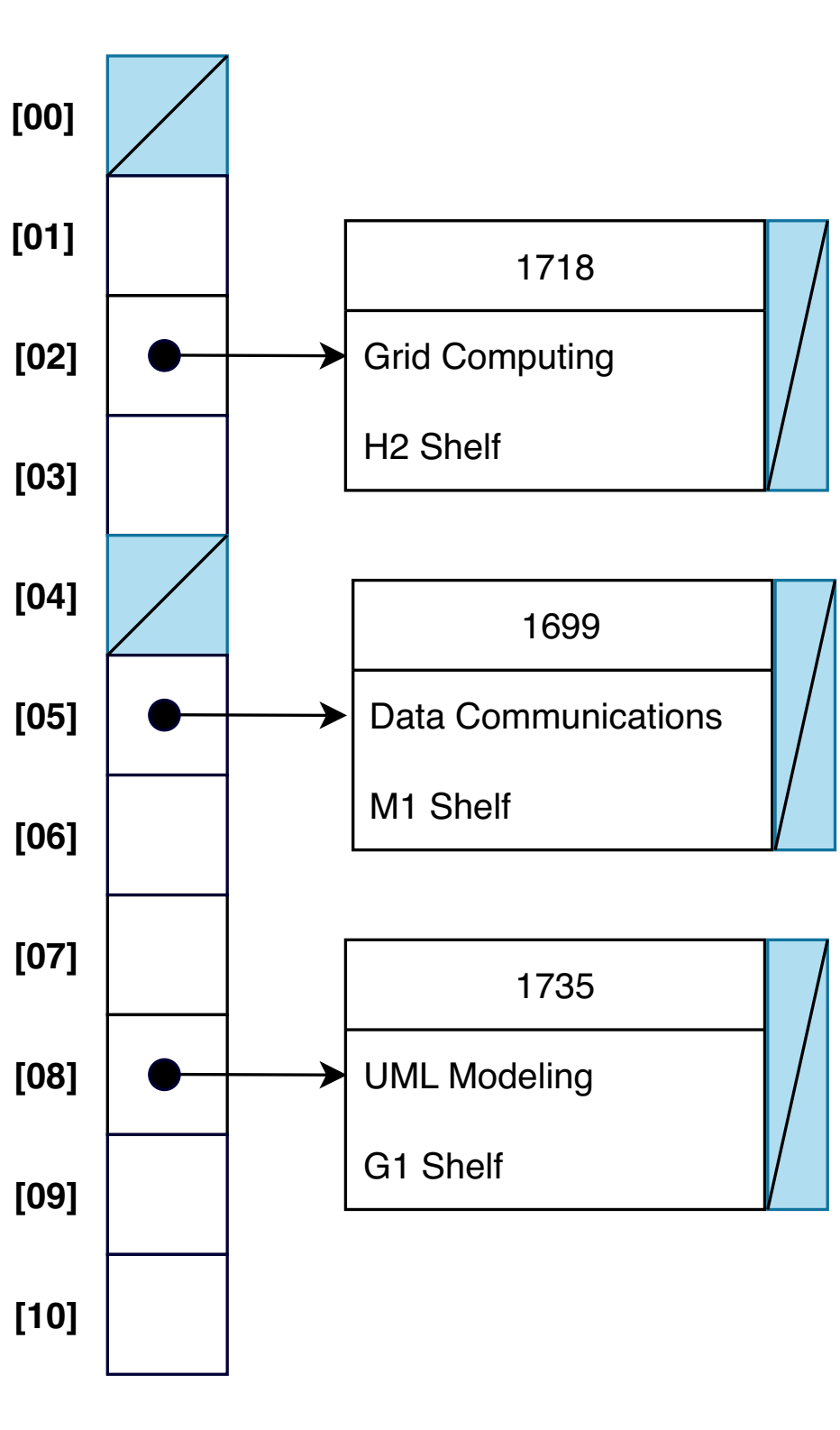
Separate chaining offers a rather simpler chaining mechanism to resolve collisions. Each slot in the hash table points to a separate data structure, such as a linked-list. This linked-list or chain stores multiple elements that share the same hash index. When a collision occurs, new elements are simply appended to the existing list in the corresponding slot.

Separate chaining is an "open hashing" technique because the hash table is "open" to accommodate multiple elements within a single bucket (usually using a chain) to handle collisions. Here is the generalized conceptual diagram for the chaining method:

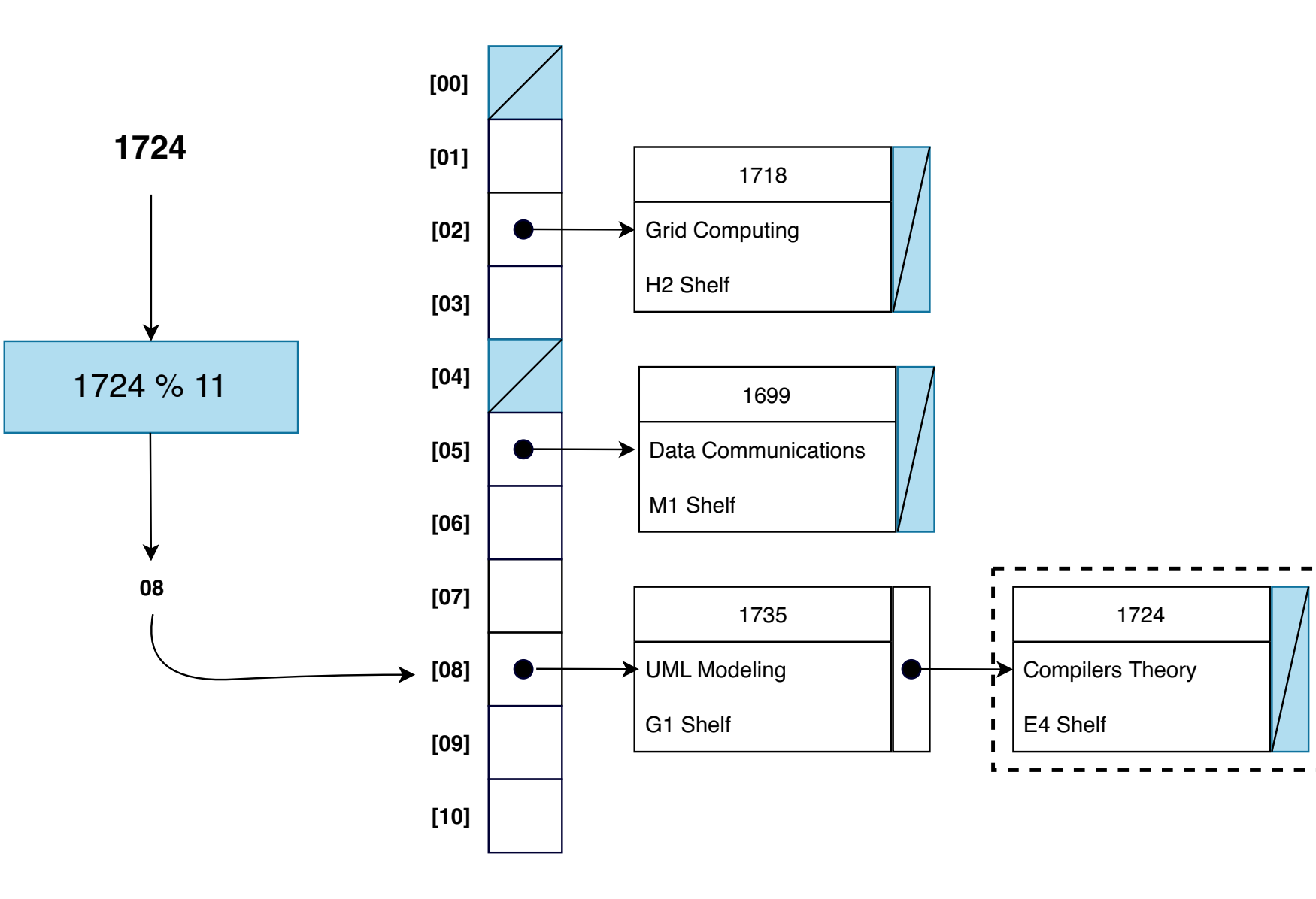


Separate Chaining

Example: Recall our earlier example of the hash table for storing books' information. Assume you have a hash table (with open hashing) of size 11 and have the following situation:



Now, here is what the hash table would look like after inserting a new book record `(1724, "Compilers Theory", "E4 Shelf")`:



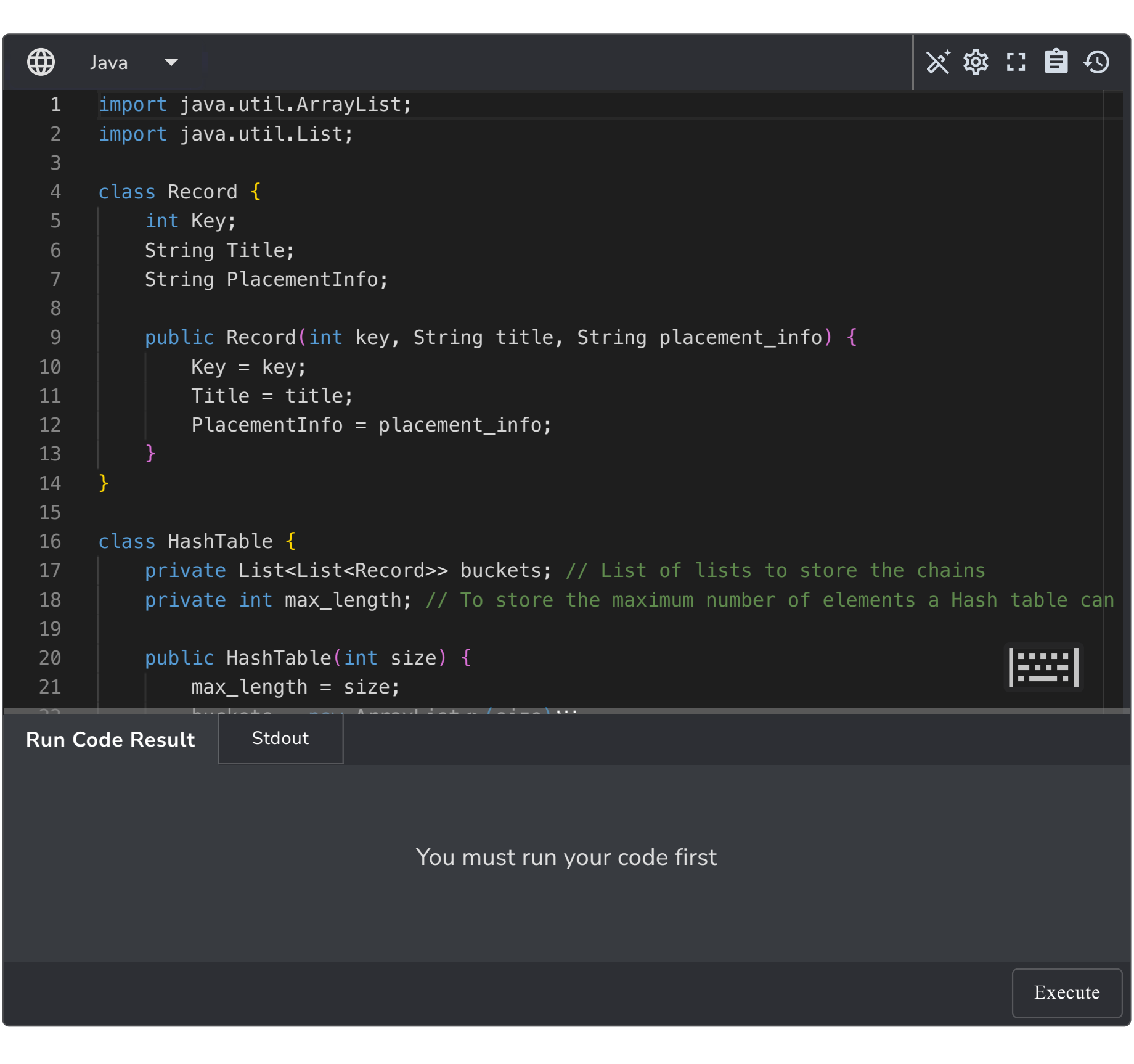
The key 1724 hashes on 08. Therefore, the item with this key is appended at the end of the chain, pointed by table slot 08.

Let's now move on to implementing the hash table with separate chaining.

A complete implementation

Insertion in a hash table with separate chaining is simple. For an item with hash key x, you need to just append the item at the list/ chain linked to the x slot of the table. Similarly, deletion operation is also more straightforward. You don't need to keep any deletion signs or marks. You can directly delete the item's node from the chain linked to the relevant hash table slot.

Here is a complete implementation of the hash table we developed for books storage:



```java
import java.util.ArrayList;
import java.util.List;

class Record {
    int Key;
    String Title;
    String PlacementInfo;

    public Record(int key, String title, String placement_info) {
        Key = key;
        Title = title;
        PlacementInfo = placement_info;
    }
}

class HashTable {
    private List<List<Record>> buckets; // List of lists to store the chains
    private int max_length; // To store the maximum number of elements a Hash table can

    public HashTable(int size) {
        max_length = size;
```

You must run your code first

Perks of separate chaining

Separate chaining has the following perks over the closed hashing techniques:

1. **Dynamic Memory Usage**: Insertions append new nodes at the chains. Unlike closed hashing, where we just put deletion mark, deleting items causes their nodes to completely removed from the chain. Thereby, the table with separate chaining can grow and shrink as per number of elements.
2. **Simple Implementation**: Implementing separate chaining is straightforward, using linked lists to manage collisions, making the code easy to understand and maintain.
3. **Graceful Handling of Duplicates**: This technique gracefully handles duplicate keys, allowing multiple records with the same key to be stored and retrieved accurately.

Downsides of separate chaining

Separate chaining has the following downsides:

1. **Increased Memory Overhead:** Separate chaining requires additional memory to store pointers or references to linked lists for each bucket, leading to increased memory overhead, especially when dealing with small data sets.
2. **Cache Inefficiency:** As separate chaining involves linked lists, cache performance can be negatively impacted due to non-contiguous memory access when traversing the lists, reducing overall efficiency.
3. **External Fragmentation:** The dynamic allocation of memory for linked lists can lead to external fragmentation, which may affect the performance of memory management in the system.
4. **Worst-Case Time Complexity:** In the worst-case scenario, when multiple keys are hashed to the same bucket and form long linked lists, the time complexity for search, insert, and delete operations can degrade to O(n), making it less suitable for time-critical applications.
5. **Memory Allocation Overhead:** Dynamic memory allocation for each new element can add overhead and might lead to performance issues when the system is under high memory pressure.

Handling Overflows

Closed hashing techniques like linear probing experience overflow when entries fill up the fixed hash table slots. Overflow can loosely indicate that the table has exceeded the suitable load factor.

Ideally, for closed hashing, the load-factor $\alpha = n/m$ should not cross 0.5, where n is the number of entries and m is table size. Otherwise, the hash table experiences a significant increase in collisions, problems in searching, and degrading performance and integrity of the table operations.

Chaining encounters overflow when chain lengths become too long, thereby increasing the search time. The load-factor α can go up to 0.8 or 0.9 before performance is affected.

Resizing the hash table can help alleviate the overflow issues. Let's explore what resizing is and when it is suitable to do.

Resizing

Resizing is increasing the size of the hash table to avoid overflows and maintain certain load-factor. Once the load-factor of the hash table increases a certain threshold (e.g., 0.5 for closed hashing and 0.9 for chaining), resizing gets activated to increase the size of the table.

When resizing, do the old values remain in the same place in the new table? The answer is "No." As resizing changes the table size, the values must be **rehashed** to maintain the correctness of the data structure.

Rehashing

Rehashing involves applying a new hash function (s) to all the entries in a hash table to make the table more evenly distributed. In context to resizing, it means recalculating hashes (according to the new table size) of all the entries in the old table and re-inserting those in the new table. Rehashing takes $O(n)$ time for n entries.

After rehashing, the new distribution of entries across the larger table leads to fewer collisions and improved performance. We perform rehashing periodically when the load-factor exceeds thresholds or based on metrics like average chain length.

Resizing and Rehashing Process:

1. Determine that the load-factor has exceeded the threshold (e.g., alpha > 0.5) and that the hash table needs resizing.
2. Create a new hash table with a larger capacity (e.g., double the size of the original table).
3. Iterate through the existing hash table and rehash each element into the new one using the primary hash function with the new table size.
4. After rehashing all the elements, the new hash table replaces the old one, and the old table is deallocated from memory.

Selecting a Hash Function

Throughout the hashing portion, we discussed only one or two hash functions. The question arises, how to develop a new hash function? Is there any technique to make your customized hash function? Let's find out the answers.

But first, we must know how to distinguish between a good and a bad hash function. Here are some ways to explain the characteristics of a good hash function in simple terms:

Characteristics of a good hash function:

Here are some characteristics that every good hash function must follow: -

1. **Uniformity and Distribution**: The hash function should spread out keys evenly across all slots in the hash table. It should not cram keys into only a few slots. Each slot should have an equal chance of being hashed to, like spreading items randomly across shelves. This ensures a balanced distribution without crowded clusters in some places.

2. **Efficiency**: It should require minimal processing power and time to compute. Complex and slow hash functions defeat the purpose of fast lookup. The faster, the better.

3. **Collision Reduction**: Different keys should end up getting mapped to different slots as much as possible. If multiple keys keep colliding in the same slot, the hash table operations will deteriorate time efficiency over time.

Hash function design techniques:

Here are a few of the commonly used techniques for creating good hash functions:

Division method

The division method is one of the simplest and most widely used techniques to compute a hash code. It involves calculating the remainder obtained by dividing the key by the size of the hash table (the number of buckets). The remainder is taken as the hash code. Mathematically, the division method is expressed as: $hash\_key = \text{key} \ \% \ \text{table\_size}$.

The division method is simple to implement and computationally efficient. However, it may not be the best choice if the key distribution is not uniform or the table size is not a prime number, which may lead to clustering.

Folding method

The folding method involves dividing the key into smaller parts (subkeys) and then combining them to calculate the hash code. It is particularly useful when dealing with large keys or when the key does not evenly fit into the hash table size.

There are several ways to perform folding:

- Digit sum:

Split the key into groups of digits, and their sum is taken as the hash code. For example, you can hash the key 541236 using digit sum folding into a hash table of size 5 as: $hash\_key = (5+4+1+2+3+6)\%5 = 21\%5 = 1$.

- Reduction:

Split the key in folds and use a reduction using any associative operation like XOR to reduce folds to a single value. Now, pass this single value through the simple division hash function to generate the final hash value.

For example, suppose you want a 12-digit key 593048892357 to be hashed onto a table of size 11113. In folding with reduction, you will split it into 3 parts of 4 digits each: 5930, 4889, 2357. Then, you XOR (⊕) the parts and pass through an ordinary hash function as: $hash\_key = (5930 \oplus 4889 \oplus 2357)\%table\_size = 7041$.

We can also add the parts: $hash\_key = (5930 + 4889 + 2357)\%table\_size = 13176\%11113 = 2063$

The folding method can handle large keys efficiently and provides better key distribution than the division method. It finds common applications where the keys are lengthy or need to be split into subkeys for other purposes.

Mid-square Method:

The mid-square method involves squaring the key, extracting the middle part, and then using it as the hash code. This technique is particularly useful when keys are short and do not have patterns in the lower or upper digits. The steps for calculating mid-square are as follows:

1. Square the key.
2. Extract the K middle digits of the square.
3. Apply simple division on these middle digits to get the final hash.

For example, consider the key 3729, and we want to hash it into a hash table of size 10.

1. Square the key: $3729 \times 3729 = 13935241$.
2. Extract the middle digits to get the hash value: 935.
3. Calculate the hash index: $H(3729) = 935\%10 = 5$.

Therefore, the key 3729 is hashed into the hash table at index 5.

The mid-square method is easy to implement and works well when the key values are uniformly distributed, providing a good spread of hash codes. However, it may not be suitable for all types of keys, especially those with patterns or significant leading/trailing zeros.

In the next section, we will solve some coding interview problems related to Hash Tables.

← Previous
Introduction To Hash Tables
Next →
Problem 1: First Non-Repeating Character (Easy)
☐ Mark as Completed